BEE 4750 Homework 5: Mixed Integer and Stochastic Programming

Name:Claire Jacobson

ID:cpj35

Due Date

Thursday, 12/04/24, 9:00pm

Overview

Instructions

- In Problem 1, you will use mixed integer programming to solve a waste load allocation problem.
- In Problem 2, you will formulate a stochastic optimization problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
import Pkg; Pkg.add("CSV")
```

```
  Activating project at `~/hw5-claire_hw5`
   Resolving package versions...
  No Changes to `~/hw5-claire_hw5/Project.toml`
  No Changes to `~/hw5-claire_hw5/Manifest.toml`
```

```julia
using JuMP
using HiGHS
using CSV
using DataFrames
using GraphRecipes
using Plots
using Measures
using MarkdownTables
```

Problems (Total: 30 Points)

Problem 1 (24 points)

Three cities are developing a coordinated municipal solid waste (MSW) disposal plan. Three disposal alternatives are being considered: a landfill (LF), a materials recycling facility (MRF), and a waste-to-energy facility (WTE). The capacities of these facilities and the fees for operation and disposal are provided below.

- **LF**: Capacity 200 Mg, fixed cost \$2000/day, tipping cost \$50/Mg;
- **MRF**: Capacity 350 Mg, fixed cost \$1500/day, tipping cost \$7/Mg, recycling cost \$40/Mg recycled;
- **WTE**: Capacity 210 Mg, fixed cost \$2500/day, tipping cost \$60/Mg;

The MRF recycling rate is 40%, and the ash fraction of non-recycled waste is 16% and of recycled waste is 14%. Transportation costs are \$1.5/Mg-km, and the relative distances between the cities and facilities are provided in the table below.

City/Facility	Landfill (km)	MRF (km)	WTE (km)
1	5	30	15
2	15	25	10
3	13	45	20
LF	-	32	18
MRF	32	-	15
WTE	18	15	-

The fixed costs associated with the disposal options are incurred only if the particular disposal option is implemented. The three cities produce 100, 90, and 120 Mg/day of solid waste, respectively, with the composition provided in the table below.

Component	% of total mass	Combustion ash (%)	MRF Recycling rate (%)
Food Wastes	15	8	0
Paper & Cardboard	40	7	55
Plastics	5	5	15
Textiles	3	10	10
Rubber, Leather	2	15	0
Wood	5	2	30
Yard Wastes	18	2	40
Glass	4	100	60
Ferrous	2	100	75
Aluminum	2	100	80
Other Metal	1	100	50
Miscellaneous	3	70	0

The information in the above table will help you determine the overall recycling and ash fractions. Note that the recycling residuals, which may be sent to either landfill or the WTE, have different ash content than the ash content of the original MSW. You will need to determine these fractions to construct your mass balance constraints.

Reminder: Use `round(x; digits=n)` to report values to the appropriate precision!

Problem 1.1

Based on the information above, calculate the overall recycling and ash fractions for the waste produced by each city.

```
recycling = (0.4*0.55 + 0.05*0.15 + 0.03*0.1 + 0.05*0.3 + 0.18*0.4 + 0.04*0.6 + 0.02*0.75 + 0
println("Recycling rate: $(round(recycling*100,digits=2))%")

ash = (0.15* 0.08 + 0.4*0.07 + 0.05*0.05 + 0.03*0.1 + 0.02*.15+ 0.05*0.02+ 0.18*0.02 + 0.04 
println("Ash content: $(round(ash*100,digits=2))%")
```

```
Recycling rate: 37.75%
Ash content: 16.41%
```

Problem 1.2

What are the decision variables for your optimization problem? Provide notation and variable meaning.

- W_{ij} : waste from city i to disposal j in Mg/day
- R_{kj} : residual waste from disposal k to disposal j
- Y_j : binary indicating whether or not disposal j is used

Problem 1.3

Formulate the objective function. Make sure to include any needed derivations or justifications for your equation(s).

$$\min_{W_{ij},Y_j} = \sum_i \sum_j 1.5(l_{ij}W_{ij} + l_{kj}R_{kj}) + \sum_j [c_j Y_j + b_j \sum_i W_{ij}]$$

Where c_j is the fixed cost for disposal j per day, l_{ij} is the distance between city i and disposal j, l_{kj} is the distance between disposal k and disposal j and b_j is the variable cost in \$/Mg for disposal j.

Problem 1.4

Derive all relevant constraints. Make sure to include any needed justifications or derivations.

- $W_{11} + W_{12} + W_{13} = 100$
- $W_{21} + W_{22} + W_{23} = 90$
- $W_{31} + W_{32} + W_{33} = 120$
- $R_{13} = 0.2(W_{11} + W_{21} + W_{31} + R_{21})$
- $R_{21} + R_{23} = 0.6(W_{12} + W_{22} + W_{32})$
- $W_{11} + W_{21} + W_{31} + R_{21} \leq 210$
- $W_{12} + W_{22} + W_{32} \leq 350$
- $W_{13} + W_{23} + W_{33} + R_{23} + R_{13} \leq 200$
- $0 \leq W_{ij}, R_{ij}$

where WTE is disposal 1, MRF is disposal 2, and Landfill is disposal 3

Problem 1.5

Find the optimal solution (using `JuMP` to solve the problem). Report the optimal objective value.

```
cities = [:C1, :C2, :C3]
disposal = [:WTE, :MRF, :LF]

waste = Dict(:C1=>100.0, :C2=>90.0, :C3=>120.0)
capacity = Dict(:WTE=>210.0, :MRF=>350.0, :LF=>200.0)
```

```julia
fixed_cost = Dict(:WTE=>2500.0, :MRF=>1500.0, :LF=>2000.0)
tipping_cost = Dict(:WTE=>60.0, :MRF=>7.0, :LF=>50.0)

recycling_cost = Dict(:MRF => 40.0)

transport_cost = 1.5

dist = Dict(
    (:C1,:WTE)=>15, (:C1,:MRF)=>30, (:C1,:LF)=>5,
    (:C2,:WTE)=>10, (:C2,:MRF)=>25, (:C2,:LF)=>15,
    (:C3,:WTE)=>20, (:C3,:MRF)=>45, (:C3,:LF)=>13
)

MRF_recycle_rate = 0.40
ash_nonrecycled = 0.16
ash_recycled_resid = 0.14
```

```
0.14
```

```julia
waste_model = Model(HiGHS.Optimizer)

@variable(waste_model, x[c in cities, f in disposal] >= 0)
@variable(waste_model, y[f in disposal], Bin)

for c in cities
    @constraint(waste_model, sum(x[c,f] for f in disposal) == waste[c])
end
for f in disposal
    @constraint(waste_model, sum(x[c,f] for c in cities) <= capacity[f]*y[f])
end

@expression(waste_model, total_cost,
    sum(fixed_cost[f]*y[f] for f in disposal) +
    sum(tipping_cost[f]*x[c,f] for c in cities, f in disposal) +
    sum(transport_cost*dist[(c,f)]*x[c,f] for c in cities, f in disposal) +
    recycling_cost[:MRF]*MRF_recycle_rate*sum(x[c,:MRF] for c in cities)
)

@objective(waste_model, Min, total_cost)
```

$2500y_{WTE} + 1500y_{MRF} + 2000y_{LF} + 82.5x_{C1,WTE} + 68x_{C1,MRF} + 57.5x_{C1,LF} + 75x_{C2,WTE} + 60.5x_{C2,MRF} + 72.5x_{C2,LF} + 90x_{C3,WTE} + 90.5x_{C3,MRF} + 69.5x_{C3,LF}$

```
set_silent(waste_model)
optimize!(waste_model)

status = termination_status(waste_model)
if status == MOI.OPTIMAL
    println("Optimal objective value: \$", round(objective_value(waste_model); digits=2))
    println("\nFacilities on/off:")
    for f in disposal
        println(f, ": ", Int(round(value(y[f]))))
    end
    println("\nWaste allocation (Mg/day):")
    for c in cities
        for f in disposal
            println(c, " -> ", f, ": ", round(value(x[c,f]); digits=2))
        end
    end
else
    println("Solver finished with status: ", status)
    println("Primal objective (if available): ", objective_value(waste_model))
end
```

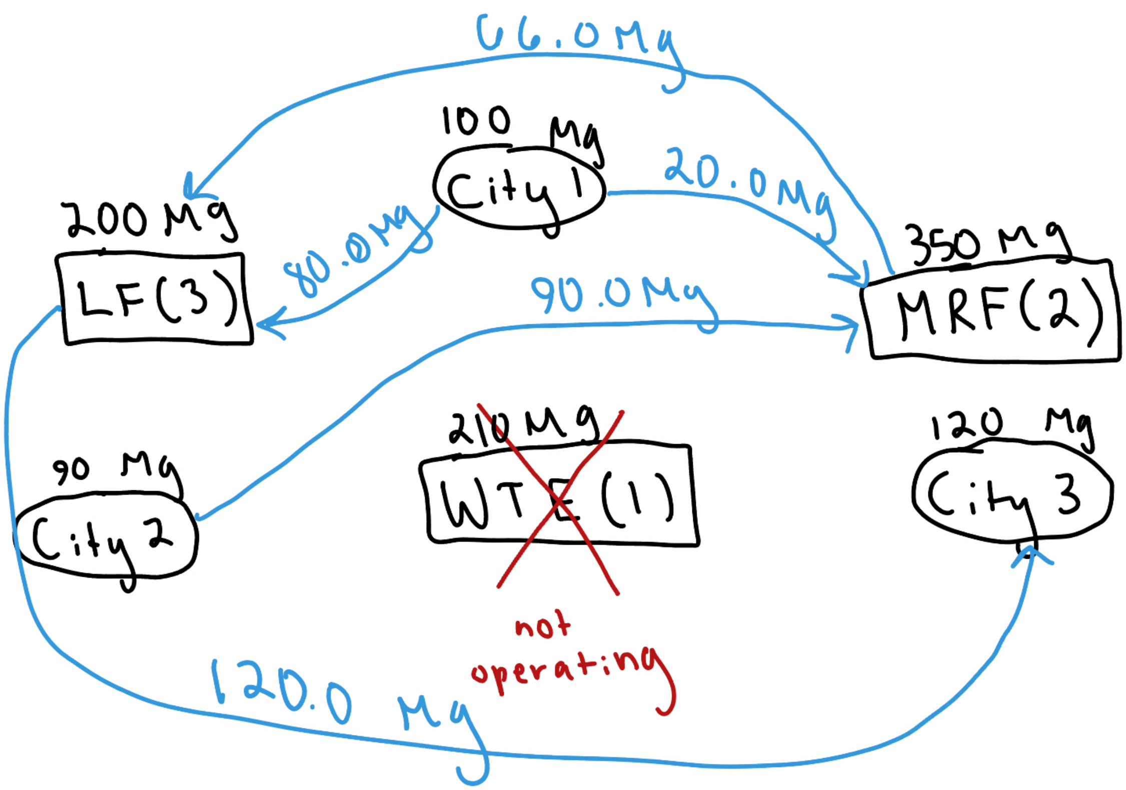
```
Optimal objective value: $23245.0

Facilities on/off:
WTE: 0
MRF: 1
LF: 1

Waste allocation (Mg/day):
C1 -> WTE: 0.0
C1 -> MRF: 20.0
C1 -> LF: 80.0
C2 -> WTE: 0.0
C2 -> MRF: 90.0
C2 -> LF: 0.0
C3 -> WTE: 0.0
C3 -> MRF: -0.0
C3 -> LF: 120.0
```

Problem 1.6

Draw a diagram showing the flows of waste between the cities and the facilities. Which facilities (if any) will not be used? Does this solution make sense?

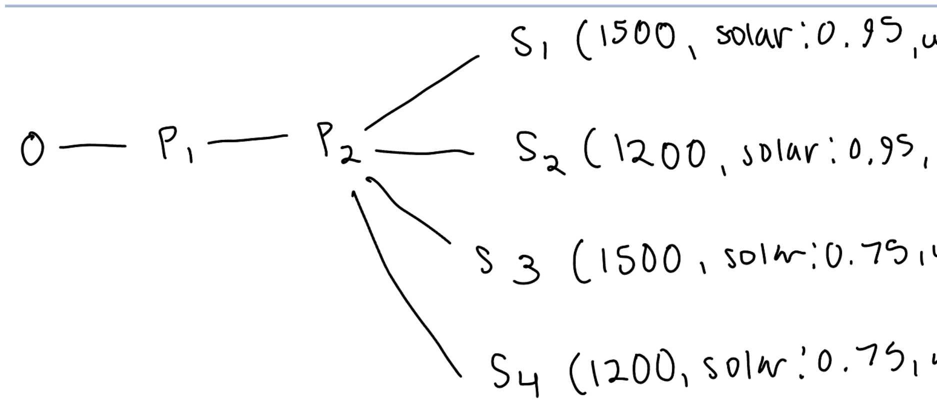


The WTE facility is not being used. This makes sense to me because the WTE facility costs the most both in set and variable cost, so if we are trying to minimize cost it makes sense that the facility would be turned off.

Problem 2 (6 points)

Consider a two-period economic dispatch problem, based on the multi-period example from Lecture 14 (on 10/29). The generator data, including ramping constraints for each generator, is provided in 'data/generators.csv.' In period 1, the demand is $d_1 = 1100$MW. In period 2, the demand is projected to be $d_2 = 1200$MW, but there is a 25% probability that it is \$1500 . In the first period, the solar capacity factor is 0.9 and the wind capacity factor is 0.45, but in the second period, there is some uncertainty: the forecasted solar and wind capacity factors are 0.95 and 0.4, respectively, but there is a 30% probability that they are 0.75 and 0.5. Your goal is to identify how to dispatch your generators to minimize the cost of meeting demand.

Problem 2.1



Draw a scenario tree for this problem.

Problem 2.2

Formulate a stochastic linear program for this problem based on your scenario tree from Problem 2.1 and the data in `data/generators.csv`.

```
gen_df = CSV.read("data/generators.csv", DataFrame)

#extracting data
gens = Symbol.(strip.(String.(gen_df.Plant)))
ng = length(gens)

#relevent parameters
Pmin = Dict(gens[i] => Float64(gen_df.Pmin[i]) for i in 1:ng)
Pmax = Dict(gens[i] => Float64(gen_df.Pmax[i]) for i in 1:ng)
varcost = Dict(gens[i] => Float64(gen_df.VarCost[i]) for i in 1:ng)
Ramp = Dict(gens[i] => Float64(gen_df.Ramp[i]) for i in 1:ng)

#resource types
resource = Dict(gens[i] => strip(String(gen_df.Resource[i])) for i in 1:ng)

# non-dispatchable wind/solar
for g in gens
    if lowercase(resource[g]) in ("wind", "solar")
```

```
        Pmin[g] = 0.0
    end
end
```

```
Warning: thread = 1 warning: parsed expected 6 columns, but didn't reach end of line around
@ CSV /Users/clairejacobson/.julia/packages/CSV/XLcqT/src/file.jl:593
Warning: thread = 1 warning: only found 6 / 7 columns around data row: 4. Filling remaining
@ CSV /Users/clairejacobson/.julia/packages/CSV/XLcqT/src/file.jl:592
Warning: thread = 1 warning: only found 6 / 7 columns around data row: 5. Filling remaining
@ CSV /Users/clairejacobson/.julia/packages/CSV/XLcqT/src/file.jl:592
Warning: thread = 1 warning: only found 6 / 7 columns around data row: 6. Filling remaining
@ CSV /Users/clairejacobson/.julia/packages/CSV/XLcqT/src/file.jl:592
```

```
#P1: deterministic
d1 = 1100.0
cf1 = Dict{Symbol,Float64}()
for g in gens
    if lowercase(resource[g]) == "wind"
        cf1[g] = 0.45
    elseif lowercase(resource[g]) == "solar"
        cf1[g] = 0.90
    else
        cf1[g] = 1.0
    end
end

# P2 (four scenarios)
demand_vals = Dict(:low => 1200.0, :high => 1500.0)
p_d = Dict(:low => 0.75, :high => 0.25)

renew_nom = Dict(:solar => 0.95, :wind => 0.40)
renew_alt = Dict(:solar => 0.75, :wind => 0.50)
p_renew = Dict(:nom => 0.70, :alt => 0.30)

struct Scenario
    name::Symbol
    prob::Float64
    d2::Float64
    cf2::Dict{Symbol,Float64}
end

scenarios = Scenario[]
```

```julia
for (dkey, pd) in p_d
    for (rkey, pr) in p_renew
        prob = pd * pr
        d2 = demand_vals[dkey]
        cf2 = Dict{Symbol,Float64}()
        for g in gens
            if lowercase(resource[g]) == "solar"
                cf2[g] = (rkey == :nom) ? renew_nom[:solar] : renew_alt[:solar]
            elseif lowercase(resource[g]) == "wind"
                cf2[g] = (rkey == :nom) ? renew_nom[:wind] : renew_alt[:wind]
            else
                cf2[g] = 1.0
            end
        end
        push!(scenarios, Scenario(Symbol("s_$(dkey)_$(rkey)"), prob, d2, cf2))
    end
end

@assert abs(sum(s.prob for s in scenarios) - 1.0) < 1e-8


model = Model(HiGHS.Optimizer)


@variable(model, g1[g in gens] >= 0)

@variable(model, g2[g in gens, si in 1:length(scenarios)] >= 0)

@constraint(model, sum(g1[g] for g in gens) == d1)

for g in gens
    @constraint(model, Pmin[g] <= g1[g] <= Pmax[g] * cf1[g])
end

for (si, s) in enumerate(scenarios)
    @constraint(model, sum(g2[g,si] for g in gens) == s.d2)
    for g in gens
        @constraint(model, Pmin[g] <= g2[g,si] <= Pmax[g] * s.cf2[g])
        @constraint(model, g2[g,si] - g1[g] <= Ramp[g])
        @constraint(model, g1[g] - g2[g,si] <= Ramp[g])
    end
end
```

```julia
@expression(model, cost_p1, sum(varcost[g] * g1[g] for g in gens))

@expression(model, expected_cost_p2,
    sum(scenarios[si].prob * sum(varcost[g] * g2[g,si] for g in gens) for si in 1:length(scen
)
```

$$0.875 g2_{Biomass,1} + 70 g2_{Geothermal,1} + 4.0249999999999995 g2_{NGCCGT,1} + 6.6499999999999995 g2_{NGCT,1} + 0.375 g2_{Biomass,2} + 30 g2_{Geothermal,2} + 1.7249999999999999 g2_{NGCCGT,2} + 2.85 g2_{NGCT,2} + 2.6249999999999996 g2_{Biomass,3} + 209.99999999999997 g2_{Geothermal,3} + 12.074999999999998 g2_{NGCCGT,3} + 19.949999999999996 g2_{NGCT,3} + 1.125 g2_{Biomass,4} + 89.99999999999999 g2_{Geothermal,4} + 5.175 g2_{NGCCGT,4} + 8.549999999999999 g2_{NGCT,4}$$

```julia
@objective(model, Min, cost_p1 + expected_cost_p2)
set_silent(model)
optimize!(model)

println("Optimal expected cost: \$", round(objective_value(model); digits=2), "\n")
println("Period 1 dispatch (MW):")
for g in gens
    println(rpad(string(g), 15), ": ", lpad(round(value(g1[g]); digits=2),8))
end

println("\nPeriod 2 dispatch by scenario (MW):")
for (si, s) in enumerate(scenarios)
    println("\nScenario ", s.name, "  (p=", round(s.prob,digits=3), ", d2=", s.d2, ")")
    for g in gens
        println("  ", rpad(string(g),12), ": ", lpad(round(value(g2[g,si]); digits=2),8))
    end
end
```

```
Optimal expected cost: $17926.75

Period 1 dispatch (MW):
Biomass        :      0.0
Hydroelectric  :    195.0
Geothermal     :      0.0
NG CCGT        :    220.0
NG CT          :    100.0
Wind           :    135.0
Solar          :    450.0
```

```
Period 2 dispatch by scenario (MW):

Scenario s_high_nom  (p=0.175, d2=1500.0)
  Biomass     :     85.0
  Hydroelectric:    500.0
  Geothermal  :      0.0
  NG CCGT     :    220.0
  NG CT       :    100.0
  Wind        :    120.0
  Solar       :    475.0

Scenario s_high_alt  (p=0.075, d2=1500.0)
  Biomass     :    100.0
  Hydroelectric:    500.0
  Geothermal  :      0.0
  NG CCGT     :    275.0
  NG CT       :    100.0
  Wind        :    150.0
  Solar       :    375.0

Scenario s_low_nom  (p=0.525, d2=1200.0)
  Biomass     :      0.0
  Hydroelectric:    285.0
  Geothermal  :      0.0
  NG CCGT     :    220.0
  NG CT       :    100.0
  Wind        :    120.0
  Solar       :    475.0

Scenario s_low_alt  (p=0.225, d2=1200.0)
  Biomass     :      0.0
  Hydroelectric:    355.0
  Geothermal  :      0.0
  NG CCGT     :    220.0
  NG CT       :    100.0
  Wind        :    150.0
  Solar       :    375.0
```

References

List any external references consulted, including classmates.